

Tools for software development

Introduction to development tools



What will the subject be about?

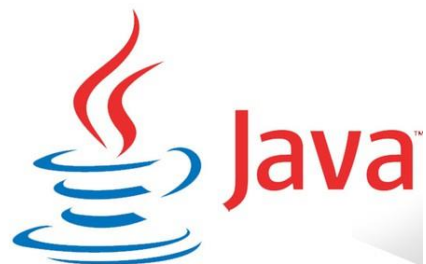
- Tool architecture overview
- Demonstration of framework development
- Test management, test design
- Types of tests
- Testing tools
- Test automation

Plan of course:

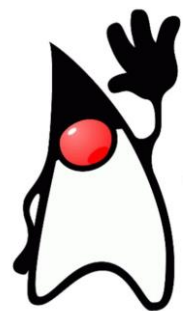
<i>Date</i>	<i>Session Type</i>	<i>Content</i>	<i>Deliverable</i>
16 Jun 2026	Introduction + Lab 1	Course overview, grading, tools, IDE, Git, project teams	team setup, repository creation, selection of application under test
18 Jun 2026	Lecture 2 + Lab 2	Testing types, unit testing, JUnit, code coverage	unit tests, coverage report, initial testing artifacts
19 Jun 2026	Lecture 3 + Lab 3	Test management, test design, manual test cases	test plan, test case template, manual tests
6 Jul 2026	Lecture 4 + Lab 4	Automated testing, Selenium	automated UI tests
7 Jul 2026	Lecture 5 + Lab 5	Test tool selection, Robot Framework	Robot Framework tests, tool evaluation decision matrix
9 Jul 2026	Lab 6 + Consultations	Postman/API testing, project finalization	Postman collection, final report, presentation
10 Jul 2026	Final Exam	Morning session dedicated to project defenses only	project defense evaluation as the final exam



Technology



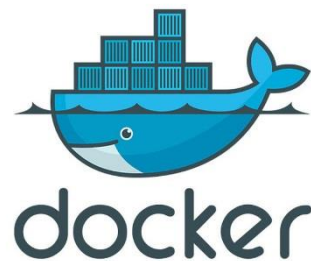
JAKARTA EE



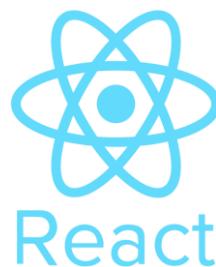
Maven™



Unit.net



docker



JUnit



POSTMAN



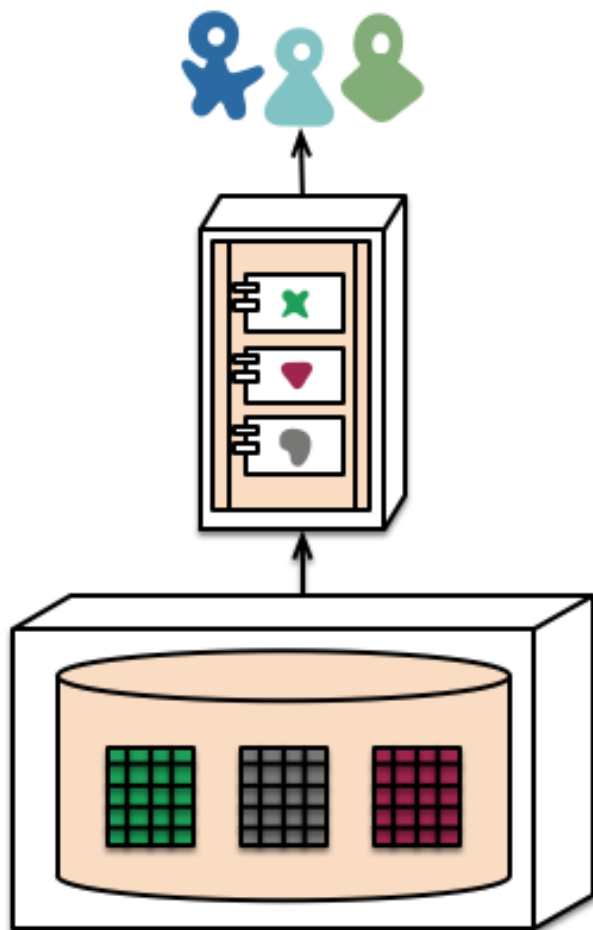
FIRST TIME WORKING WITH ARCHITECTURE SOFTWARE



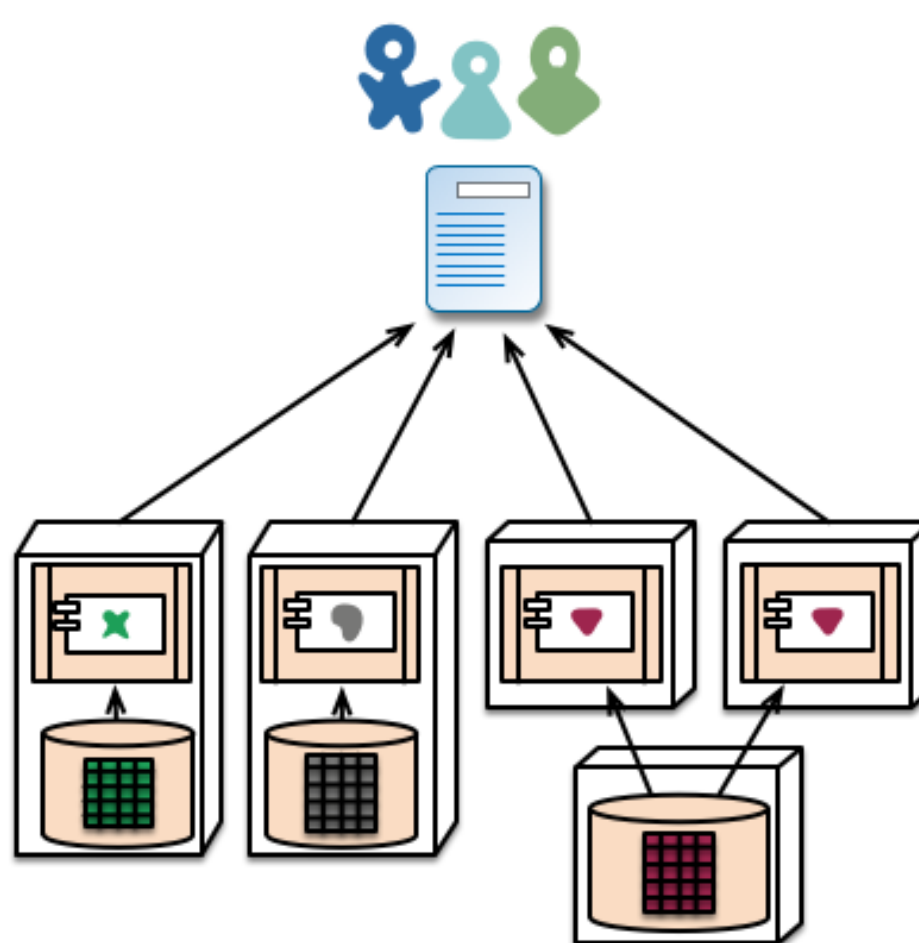
Choosing a good tool

- Modern software development
 - choice of the right tools is crucial for building **reliable**, **scalable**, and **maintainable** systems
- We need to focus on two things when choosing tools:
 - **Architecture selection:**
 - tools that help plan and design the software structure
 - **Software testing:**
 - tools that ensure the quality and correctness of the code

Change of approach in architecture programming



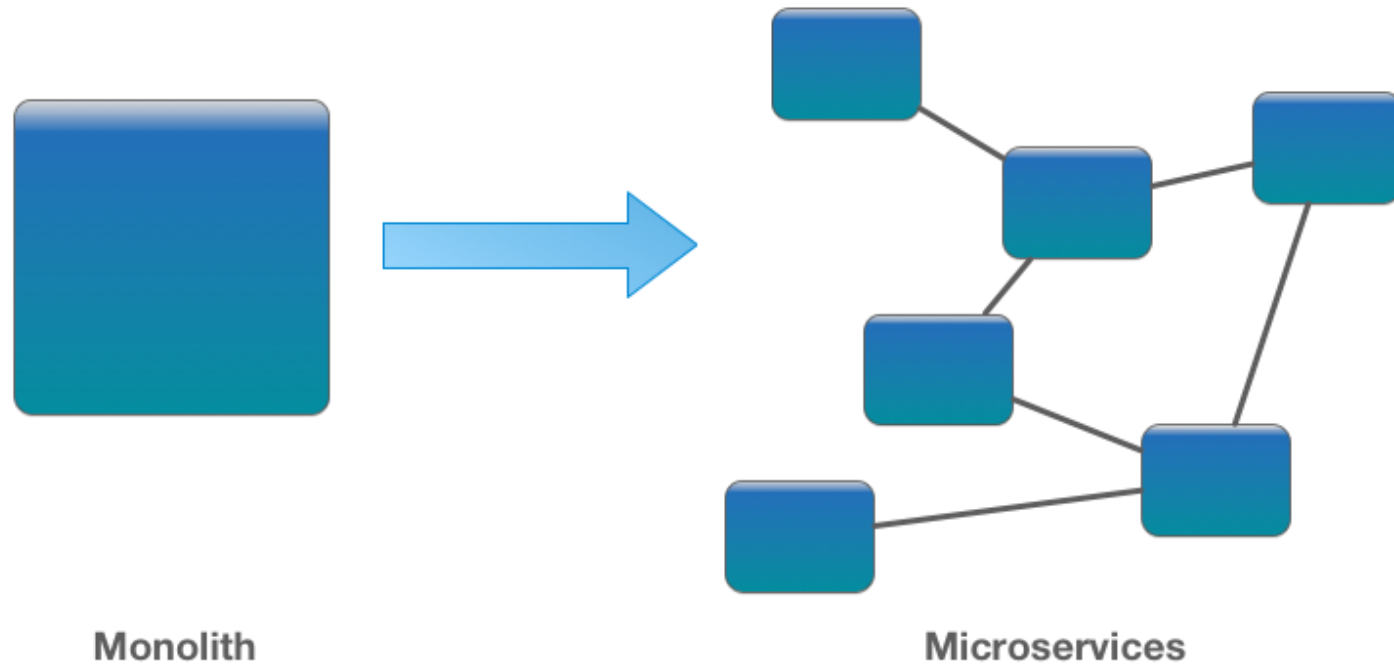
monolith - single database



microservices - application databases

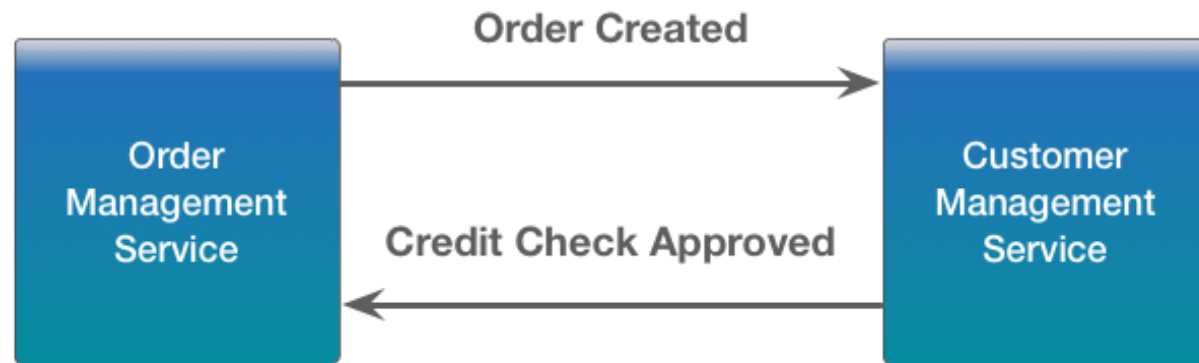
Why microservices?

- Enable faster innovation
- Businesses need to innovate much faster to stay ahead of the competition

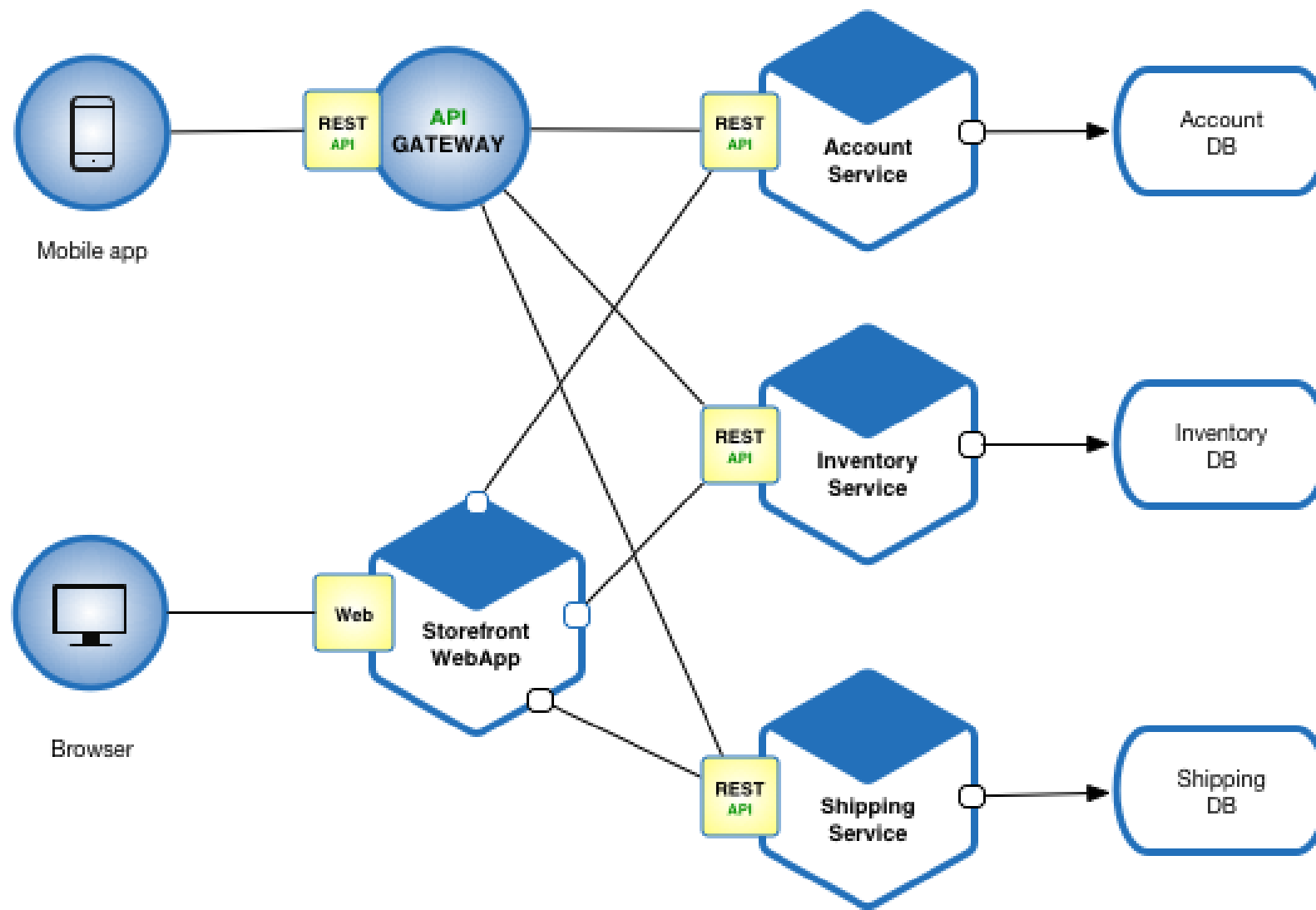


Problems or solution with microservices

- During development, the problem of managing distributed data must be solved
- Each microservice has its own private database
- Mostly using an event-driven development architecture
- Easier service updates
- Always a functional application



Example



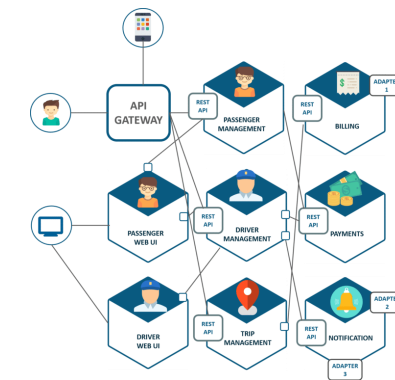
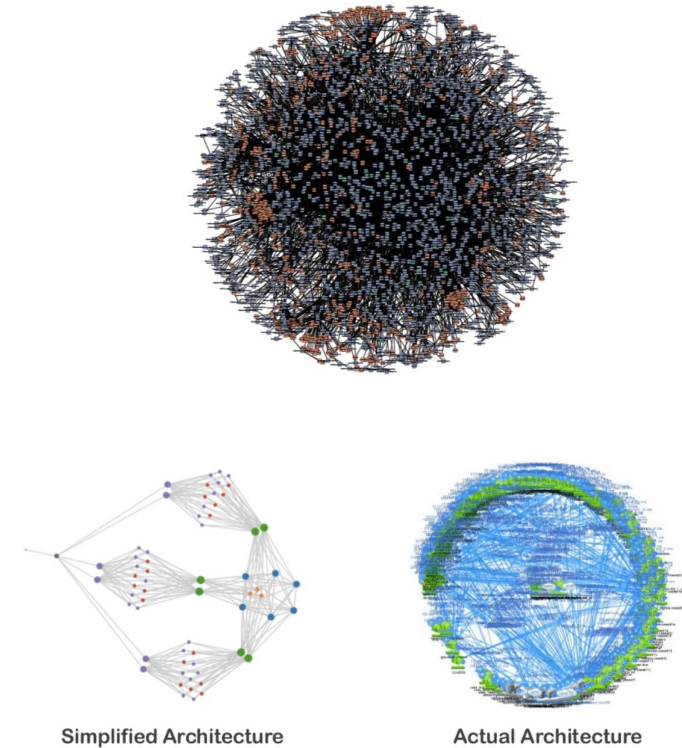
Advantages of microservices

- Enables continuous delivery and deployment of large and complex applications:
 - Testability
 - Deployability
 - Organizing a development team
- Each microservice is relatively small:
 - Easier to understand
 - IDE works faster
 - Application starts faster
- Improving fault isolation
- Independence from technology



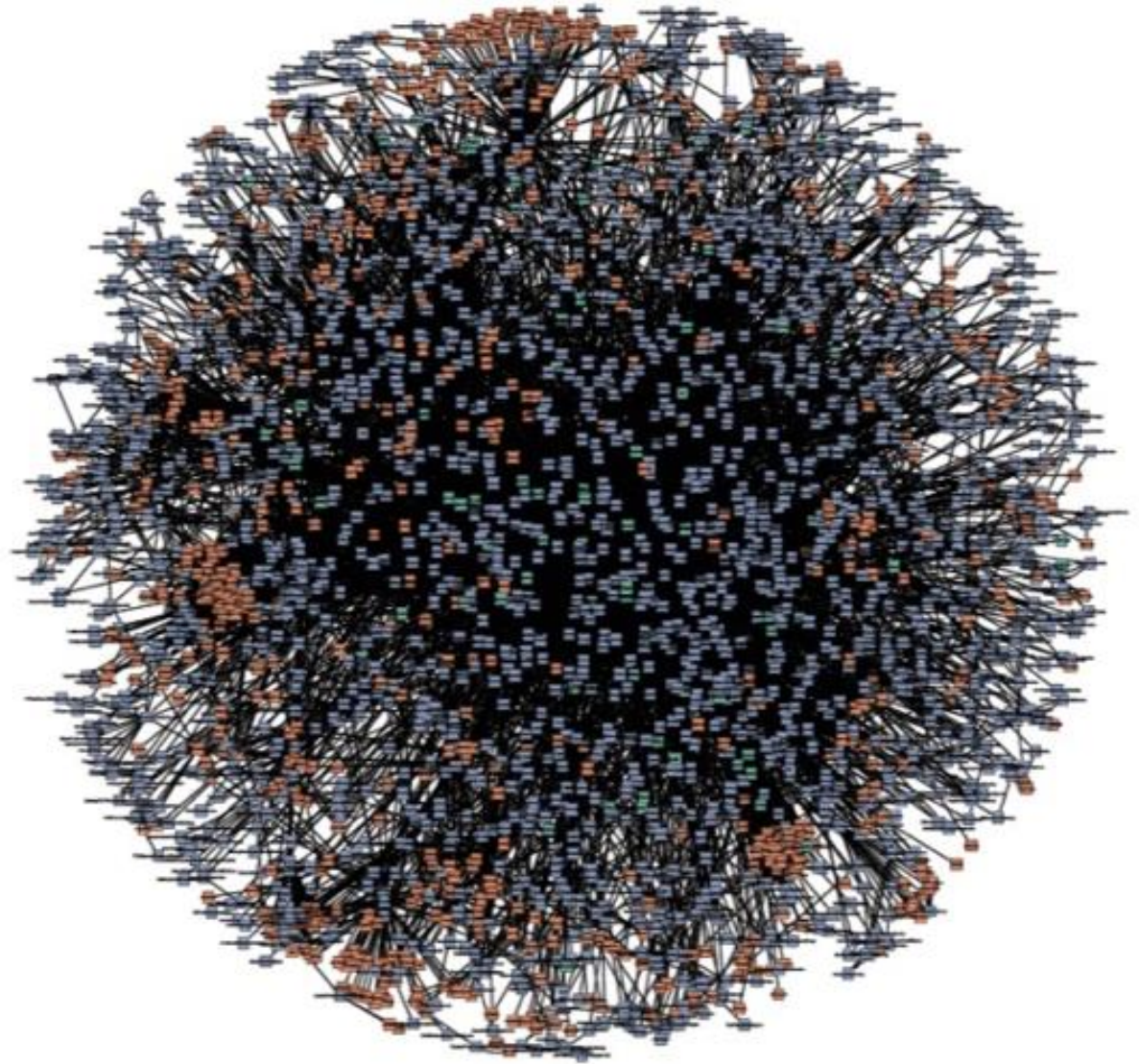
Is it used at all?

- Netflix
 - streaming service 30% of internet load
 - over 800 types of devices
 - serving a trillion calls per day
- Amazon
 - Originally a two-tier architecture
 - Migration and expansion was costly
 - Today serving 100-500 services
- Uber
 - using API gateways and independent services



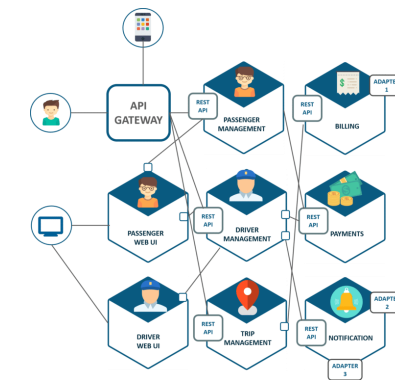
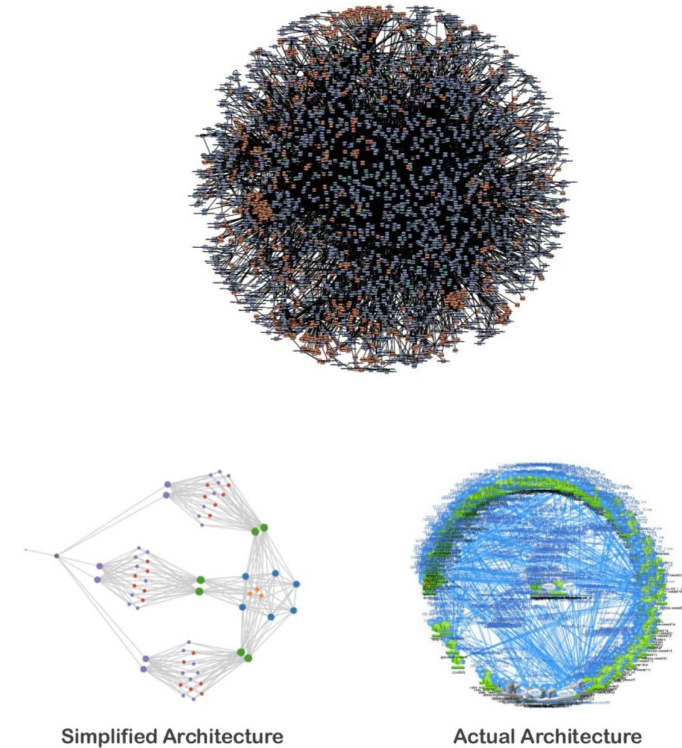
Is it used at all?

- Netflix
 - streaming service 30% of internet load
 - over 800 types of devices
 - serving a trillion calls per day
- Amazon
 - Originally a two-tier architecture
 - Migration and expansion was costly
 - Today serving 100-500 services
- Uber
 - using API gateways and independent s



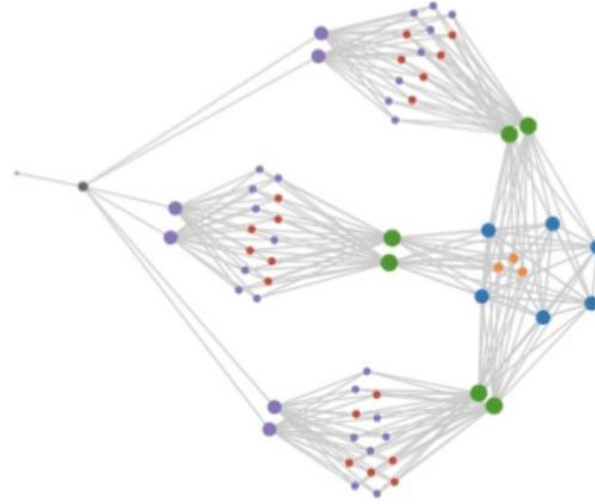
Is it used at all?

- Netflix
 - streaming service 30% of internet load
 - over 800 types of devices
 - serving a trillion calls per day
- Amazon
 - Originally a two-tier architecture
 - Migration and expansion was costly
 - Today serving 100-500 services
- Uber
 - using API gateways and independent services

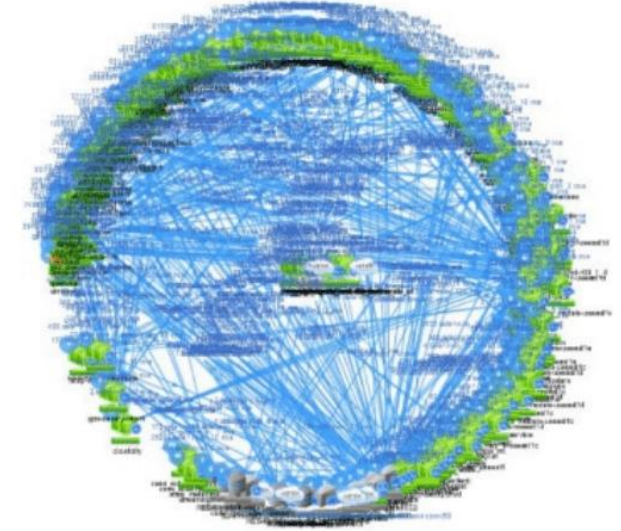


Is it used at all?

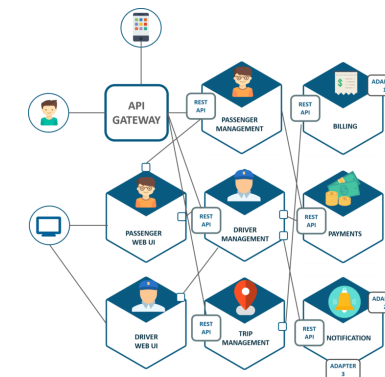
- Netflix
 - streaming service 30% of ir
 - over 800 types of devices
 - serving a trillion calls per d
- Amazon
 - Originally a two-tier architecture
 - Migration and expansion was costly
 - Today serving 100-500 services
- Uber
 - using API gateways and independent services



Simplified Architecture

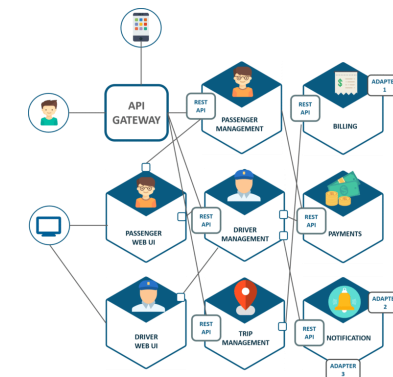
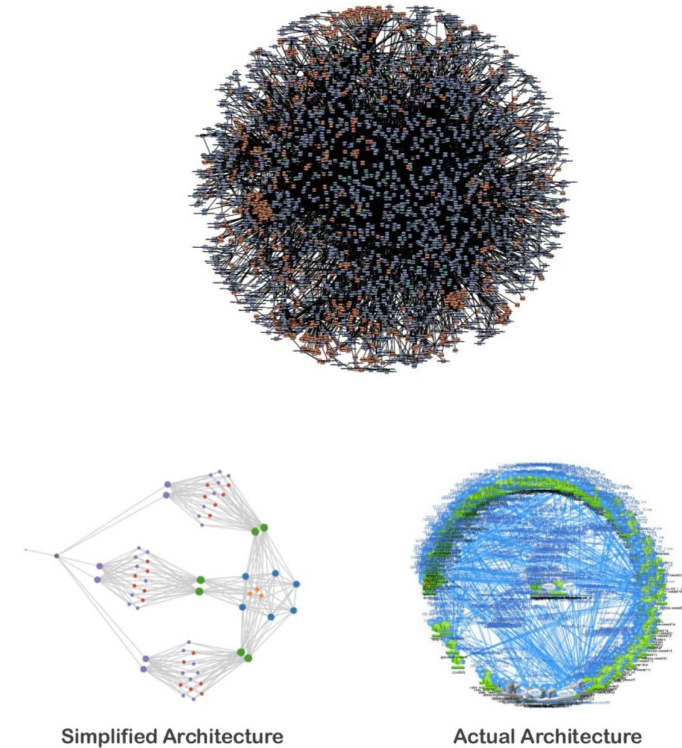


Actual Architecture



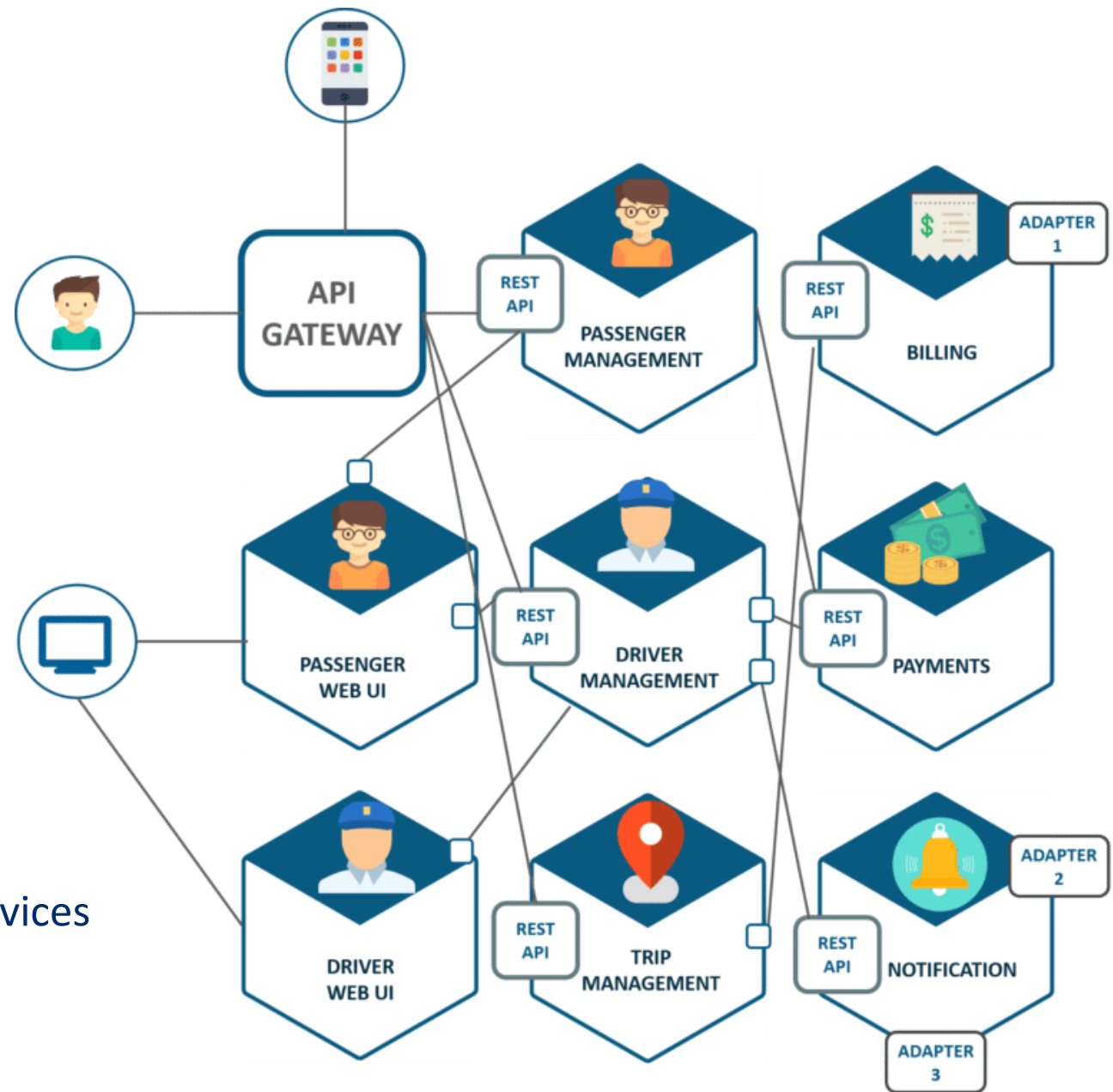
Is it used at all?

- Netflix
 - streaming service 30% of internet load
 - over 800 types of devices
 - serving a trillion calls per day
- Amazon
 - Originally a two-tier architecture
 - Migration and expansion was costly
 - Today serving 100-500 services
- Uber
 - using API gateways and independent services



Is it used at all?

- Netflix
 - streaming service 30% of internet load
 - over 800 types of devices
 - serving a trillion calls per day
- Amazon
 - Originally a two-tier architecture
 - Migration and expansion was costly
 - Today serving 100-500 services
- Uber
 - using API gateways and independent services



Another dilemma when choosing architecture



Why do software tests at all?

- Software testing originated with software development
- Increasing technology performance – increasing complexity of systems and their testing
- No static state of tests:
 - Development of new technologies
 - New programming languages
 - New approaches
 - Using mobile technology, cloud, virtualization
- It is an integral part of the software development cycle

What do we need to know...

- **Requirements:**
 - customer's wishes describing the functions and features of the system being created
- Basic types:
 - **Functional** requirements - what it should do
 - **Non-functional** requirements - features, limitations
- Not always defined correctly

What do we need to follow...

- Technical standards
 - STN, EN, CSN
- Standards
 - ISO, Information technology e.g. IEEE
- Laws
 - Given by the state
 - VAT



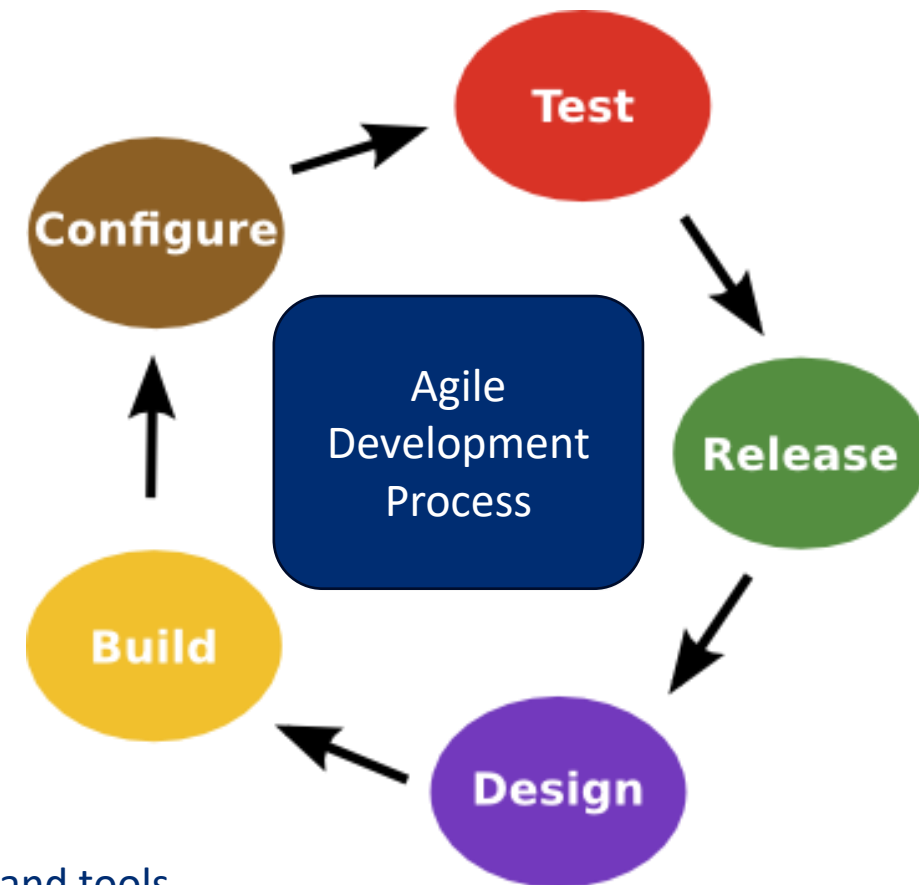
Development method and testing

- **Traditional way**

- Feasibility Study
- SW Requirements Specification
- Design Document

- **Agile development**

- Adaptability, flexibility and adaptability to change
- Manifesto:
 - People and their communication are more important than processes and tools
 - Working software is more important than cluttered documentation
 - Collaboration with the customer is at the forefront before negotiating a contract
 - It is more important to respond to changes than to stick to a plan
- Frameworks: Scrum and Extreme Programming, ...



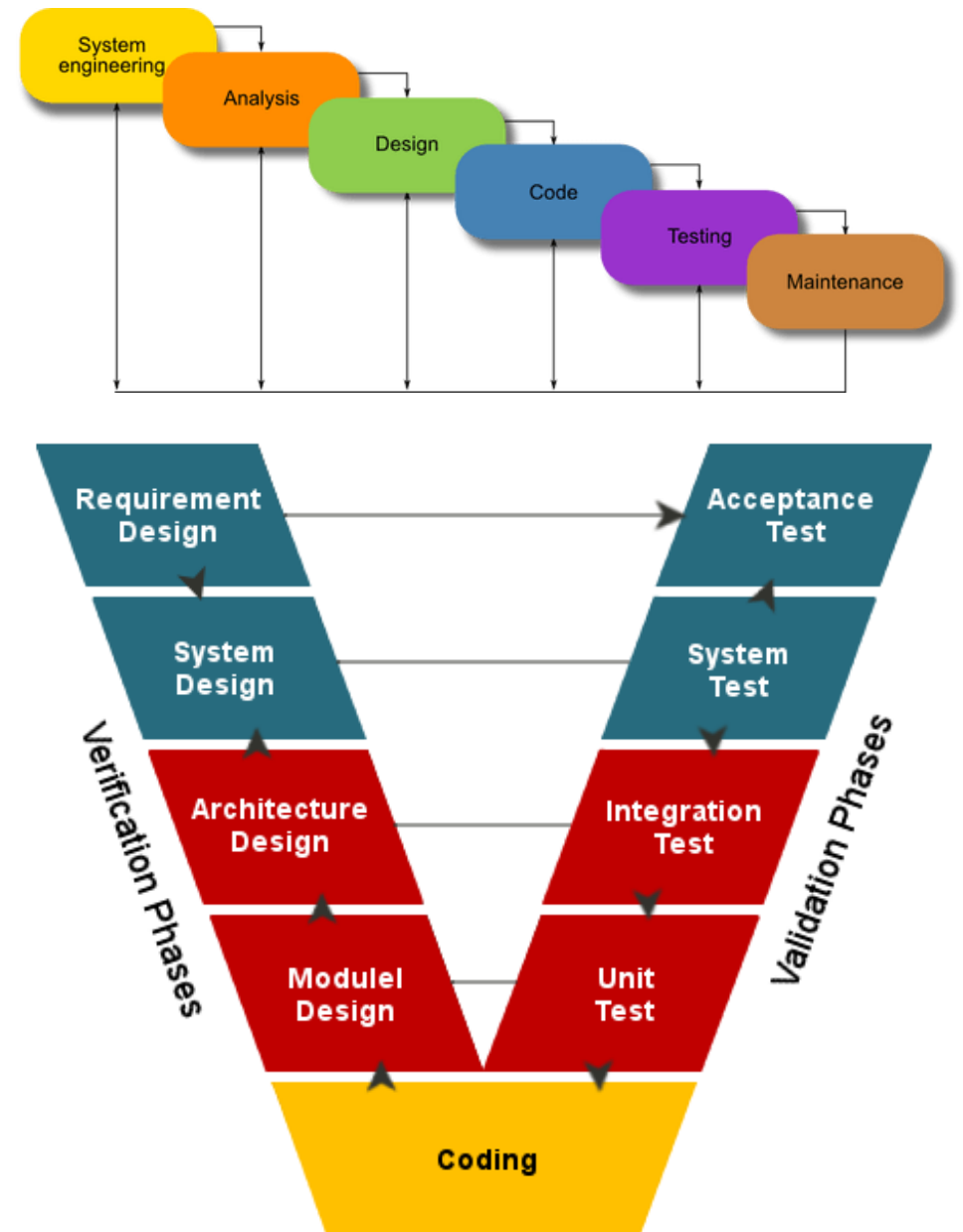
Basic concept of software quality and testing

- High-quality vs. Low-quality?
- Different Views:
 - Transcendental
 - User Production
 - Production
 - Price
- According to ISO/IEC 25010:
 - **The extent to which software meets stated and implied needs when used for its intended purpose**
 - Basic characteristics:
 - Functionality
 - Efficiency
 - Compatibility
 - Usability
 - Fault-Resistance
 - Safety
 - Sustainability
 - Portability



Verification and Validation

- Verification:
 - **Are we creating the product correctly?**
 - Requirements, design, source code, integration
 - Testing – a form of dynamic verification
- Validation:
 - **Am I creating the right product?**
 - Customer participation, functionality and service verification
 - Acceptance testing
 - delivered SW does what it is supposed to



Software Quality Assurance Techniques

- **Static analysis**

- The goal is to examine the product – source code, model, design document
- Without running

- **Dynamic analysis**

- Examining the behavior and features of a product at runtime
- Not testing
 - Memory analysis
 - System performance
 - Troubleshooting

- **Testing**

- Typical dynamic activity
- Determining functionality and overall functionality in deployment



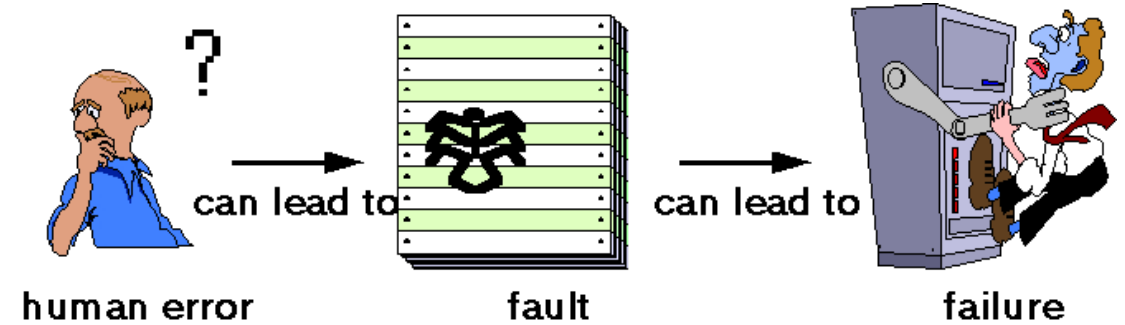
Static



Dynamic

Basic concepts

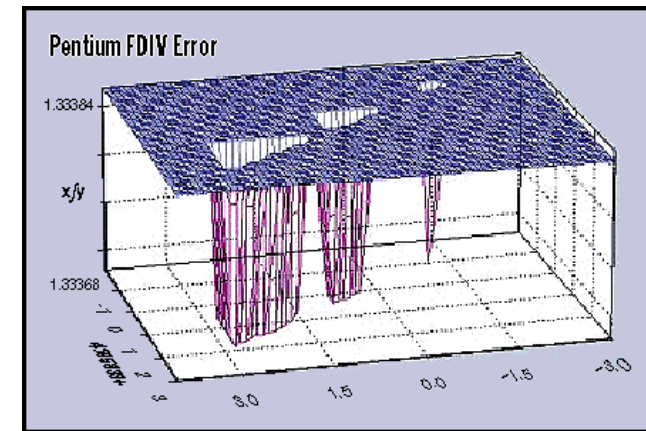
- **Dependability** – operational reliability
 - The ability of the system to deliver a service that can be trusted
- Basic threats:
 - **Faults**
 - static inoperability
 - **Errors**
 - unexpected action leading to failure
 - **Failures**
 - *If the service deviates from the correct state*



```
Static int Distance(int X1, int X2)
{
    return X1 - X2;
}
```

In case we didn't test

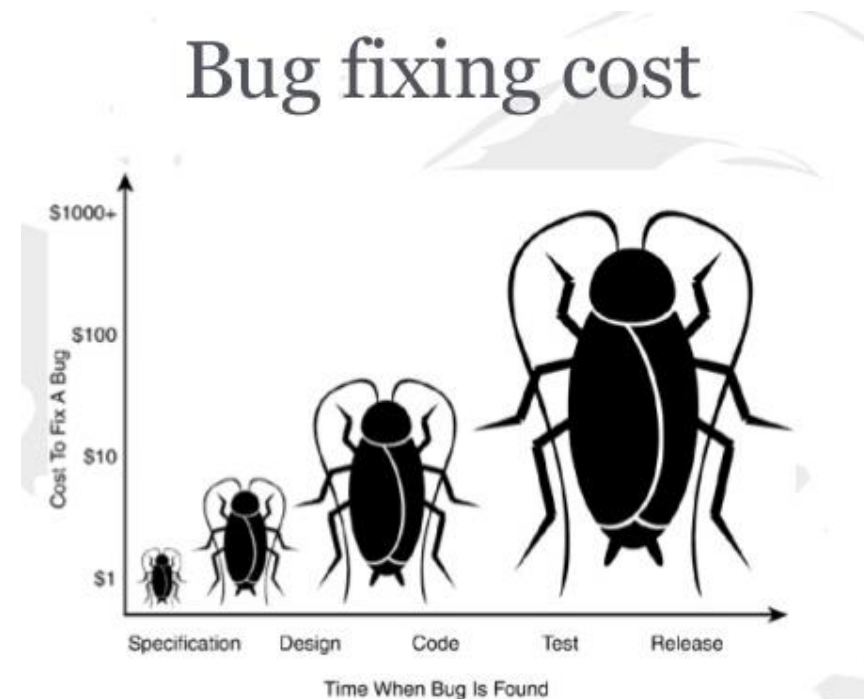
- Therac-25 (1985-1987)
- Patriot missile defense system(1991)
- Disney: The Lion King game (1995)
- Intel Pentium FDIV bug (1994)
- NASA Mars Climate Orbiter (1999)
- Y2K bug (1974)
- Rocket explosion Ariane 5, (1996)
- Warcraft III: Reforged (2020)



What is a bug?

- **Marked as a software defect**
 - Source code
 - Documentation
- **Defect Occurrence:**
 - The system does something that it is not supposed to do according to the specification
 - The system does not do what it is supposed to do
 - The system does something that is not defined
 - The system does not do something that was not specified but should have been
- **Defect severity and priority:**
 - Critical
 - High
 - Medium
 - Low

Example search on YT: Notepad in Windows XP
“Bush Hid the Facts”

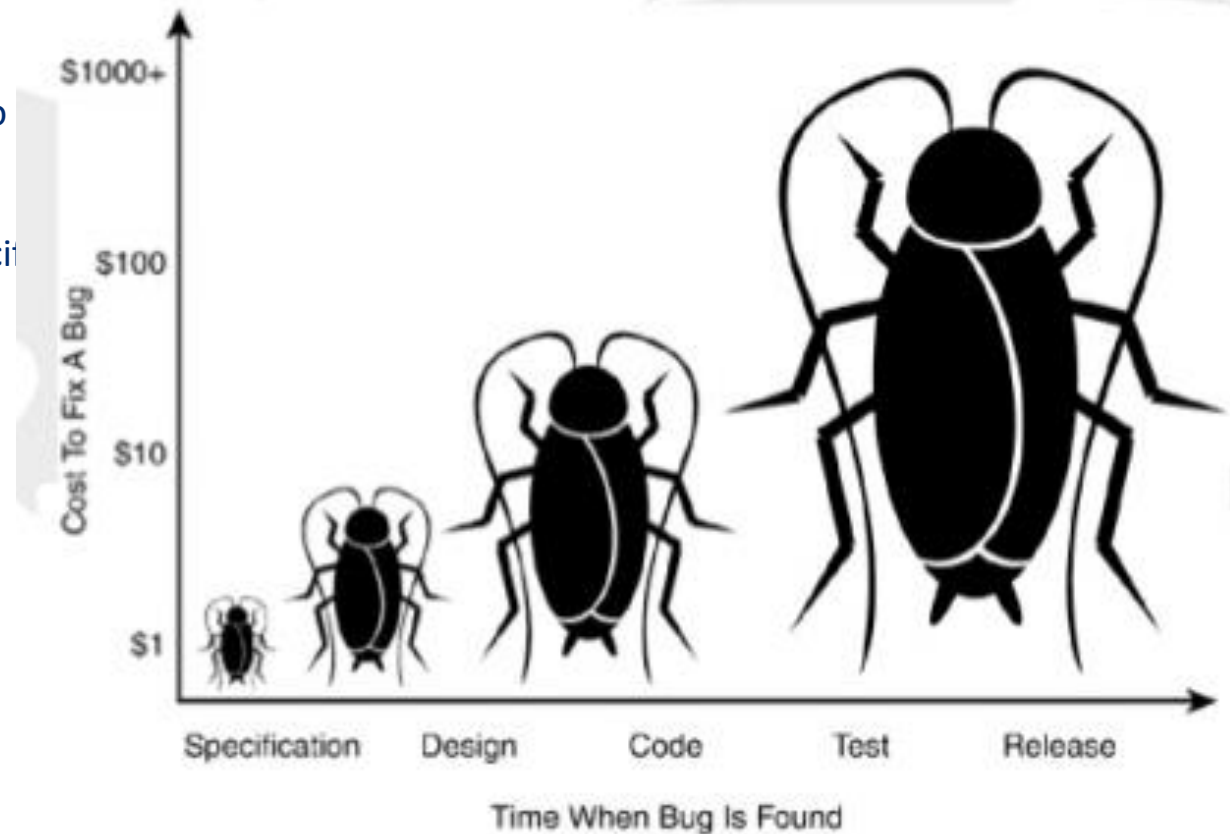


What is a bug?

- **Marked as a software defect**
 - Source code
 - Documentation
- **Defect Occurrence:**
 - The system does something that it is not supposed to
 - The system does not do what it is supposed to do
 - The system does something that is not defined
 - The system does not do something that was not specified
- **Defect severity and priority:**
 - Critical
 - High
 - Medium
 - Low

Example search on YT: Notepad in Windows XP
“Bush Hid the Facts”

Bug fixing cost



What is testing?

- The process of **controlled execution** of a software product to determine whether it meets the specified or implicit needs of the user
- Static analysis – sometimes not considered testing
- Testing examines the software:
 - Gain information about its quality
 - Meet the requirements of the client



Software Testing

The importance of testing

- Software is custom-made
- Every software is unique
- All people make mistakes
- The impossibility of improving and perfecting "software production" to an ideal form
- Tests serve precisely to verify the quality of the created software

Basic axioms of testing

[Patton 2002]

1. No non-trivial program can be completely untested
2. Testing is a matter of risk assessment
3. Testing can only prove the presence of defects
4. The more defects are found, the more of them remain in the product



Programmer vs. tester

- Why shouldn't a programmer test his own code?
- He may not understand the specification correctly
- Of course everyone tests that the code works correctly

- **Tester**

- Gets unknown code
- Tries to find the malfunction
- Without knowledge of the software, he can identify what the programmer forgot



Tester and programming

- Basic mistake
 - Testing is a simple, boring and monotonous job
 - [G. Myers 2004] Art of Software Testing
 - “Testing is an extraordinarily creative and intellectually demanding task.”
- Every good tester is able to view the system as:
 - Programmer
 - End user
 - Analyst
- Knowledge of technologies is an asset to the tester and his work
- He should have a minimum of programming knowledge
- Language basics – syntax and semantics (exception e.g. SQL)



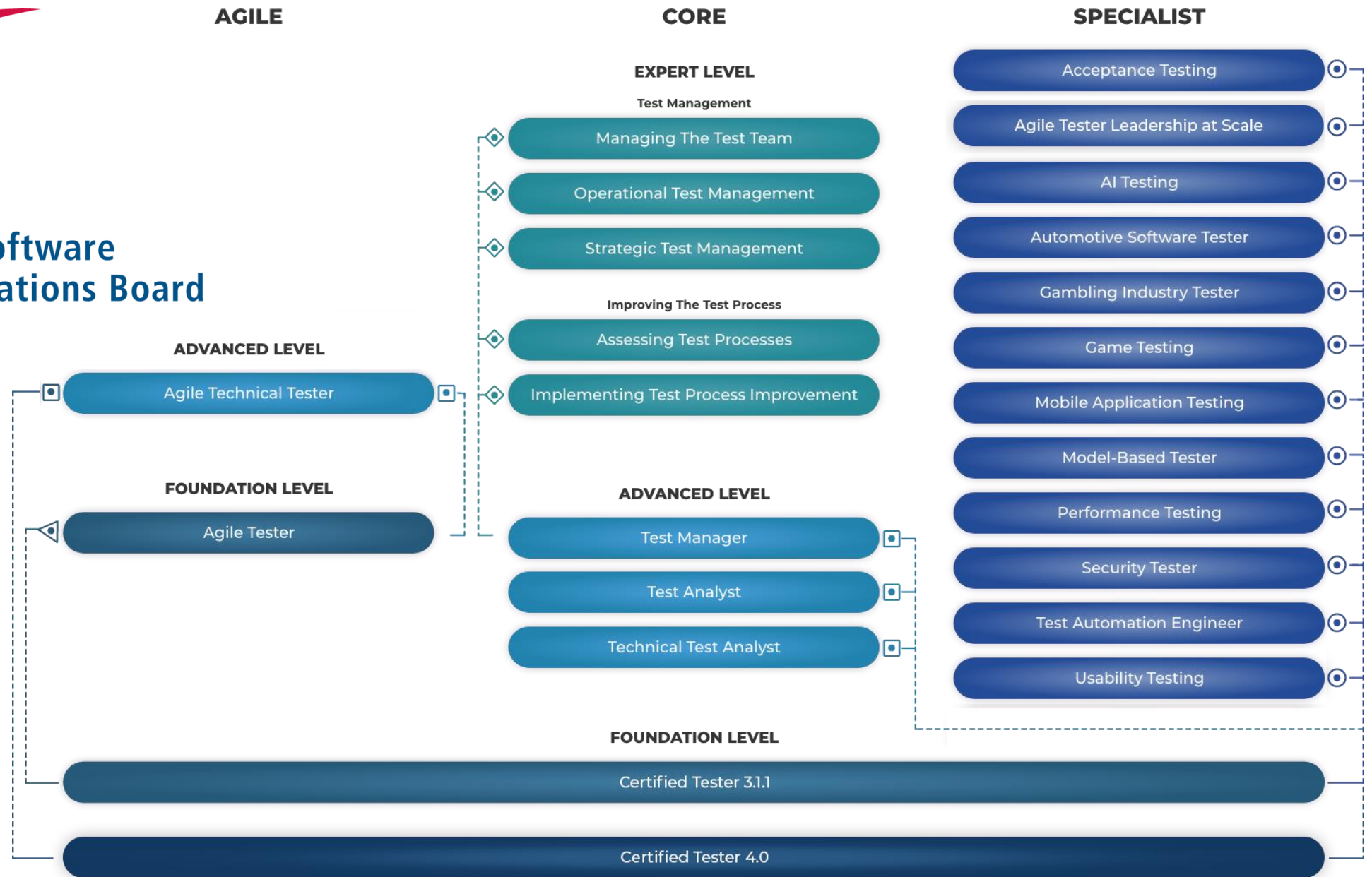
Test team roles

- Tester
- Test Analyst
- Automation Test Tester
- Test Lead
- Test Manager

Certification

- **ISTQB:** Certifying Software Testers Worldwide
 - www.istqb.org
- **CaSTB** – Czech and Slovak Testing Board
 - <https://castb.org/sk>
- **CFTL** – French Software Testing Committee
 - <https://cftl.fr/>
- Test types::
 - ISTQB Certified Tester Foundation Level (CTFL)
 - ISTQB Certified Tester Advanced Level (CTAL)
 - ISTQB Certified Tester Advanced Level (Core)
 - ISTQB Certified Tester Advanced Level (Specialist)
 - ISTQB Certified Tester Expert Level (CTEL)





SOFTWARE TESTER



What my friends think I do



What my mom thinks I do



SITE DOWN
What society thinks I do



What programmers think I do



What I think I do



What I actually do

Thank you for your attention

doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk